Blockchain Project CS4455

John Foley 22366741

Code Overview

For this ticketing system implementation I have used a React frontend alongside the use of carbon components design system. My ticketing system token is called Match Ticket (solidity file can be found at \contracts\TicketingSystem.sol). Within the project, there are a number of pages providing the different specified functionality.

The main App.js file is an entry point to other pages, containing some variables that are passed and remembered from page to page, such as the user's wallet address.

Create Wallet Page (\src\pages\CreateWallet.jsx)

- Two password text inputs take in a password and confirmation password that must be over 8 characters long.
- On clicking the Generate Wallet button, a generateWallet() function is called. This function verifies password length, creates a wallet and encrypts the wallet using the specified password.
- The wallet address and private key are displayed thanks to UseState variables and a Download Wallet Keystore button allows users to download the encrypted wallet keystore.

Check Balance Page (\src\pages\CheckBalance.jsx)

- A text input takes in the wallet address the user wishes to check the balance of.
- The checkBalance function is called. This function connects to the Sepholia test network, checks if the input address is valid and then checks the balances.
- The page then renders the SETH balance of the user, the ticket token balance of the user and the ticket token balance of the address. This shows the remaining amount of tickets that can be bought and is a useful feature for the venue and bouncers.

Purchase Ticket Page (\src\pages\PurchaseTicket.jsx)

- This page is used to buy a ticket token directly from the smart contract. Text inputs show cost per tickets and total costs.
- The user can buy a ticket using an encrypted keystore file and password, or a private key and wallet address.
- On clicking the purchase ticket button either purchaseTicketViaKeystore or purchaseTicketViaPrivateKey are called. Both these functions create a buyToken transaction and add a value based on the amount of tickets the user specified to purchase.
- The transaction is then signed and sent to the Sepolia testnet. After this the approveTickeyTransfer function is called

- This function creates an approve transaction, allowing a specified doorman account to transfer tickets back to the doorman account. This approve transaction facilitates the ability of a doorman to essentially redeem a ticket and send it to the doorman account without the need of the owner's private key.

<u>Transfer Ticket Page (\src\pages\TransferTicket.jsx)</u>

- This page allows users to transfer tickets. This can be a transfer to friends, to the contract or to the doorman. This functionality requires the owner's private key which differentiates it from the redeem ticket page.
- Text inputs take in the recipient's address as well as the amount of tickets to send. The page supports both keystore file and password or wallet and private key.
- On clicking the transfer ticket button either transferTicketViaKeystore or transferTicketViaPrivateKey is called.
- These functions create transfer transactions, sign them using the user's private key and send the transaction to the Sepolia testnet.

<u>Redeem Ticket Page (\src\pages\RedeemTicket.jsx)</u>

- This page is to be used by a doorman to redeem tickets on behalf of the user
- Inputs take in the amount of tickets to redeem, the user address who is redeeming and the private key of the doorman
- On clicking the redeem ticket button, the redeemTicket function is called.
- This function creates a transferFrom transaction, transferring a ticket from the user address to the doormans address. This is possible as the user has approved this on purchasing the ticket.
- The transaction is signed off with the doormans private key and a ticket is ‘redeemed’.

Design Description Detail

Some key design details when deploying my contract and implementing the front end include:

1. Starting the contract with the initial supply of tokens instead of the address that deployed the contract like in previous tutorial contracts.
2. The contract with a 0.0001 ether cost per ticket. The contract has a max purchase of 99 tickets per transaction, to avoid one user buying the entire supply in a single transaction.
   `'require(msg.value < 0.01 ether, "Too much ETH sent; Can only buy 99 tickets per transaction");'`
3. There is a wallet address dedicated as the “doorman account”. This account is where tickets redeemed by the doorman are sent to. An alternative approach would be for the doorman to send tickets back to the contract, but then tickets could be re-purchased leading to tickets being oversold.
4. On purchase of a ticket, the buyer also creates an approve transaction which allows the doorman to redeem a ticket making use of the transferFrom function. Redeeming a ticket in essence transfers a ticket to the “doorman account” from the buyer's account without

the need of the buyer's private key. It does however require the private key of the doorman account, meaning only the doorman can redeem tickets.

5. There is also the option for a manual transfer of tickets. This can be used to transfer a ticket from friend to friend, transfer a ticket to the doorman account or return tickets back to the contract for others to buy again.
6. The webpage supports the transfer of tickets as well as the purchase of tickets via a keystore JSON file and password or via a wallet address and private key. This is for ease of use and to not need to use a private key. Having a keystore is safer than storing a private key.

Successful Transaction Links

Initially I had worked with contract 0x0cce4E7d5DE14Ce4efd7Dd4983ab26B8e9aaeA3F , until I decided to redeploy with minor changes to the contract such as cost per ticket and maximum purchases. There are many successful transactions on this old contract as well as the newly deployed contract at 0x94dCB7c12F368D58a81C0C1Eb0d98F5a77ec3591. The documented transactions below belong to the newly deployed contract.

Deployment of Match Ticket Contract

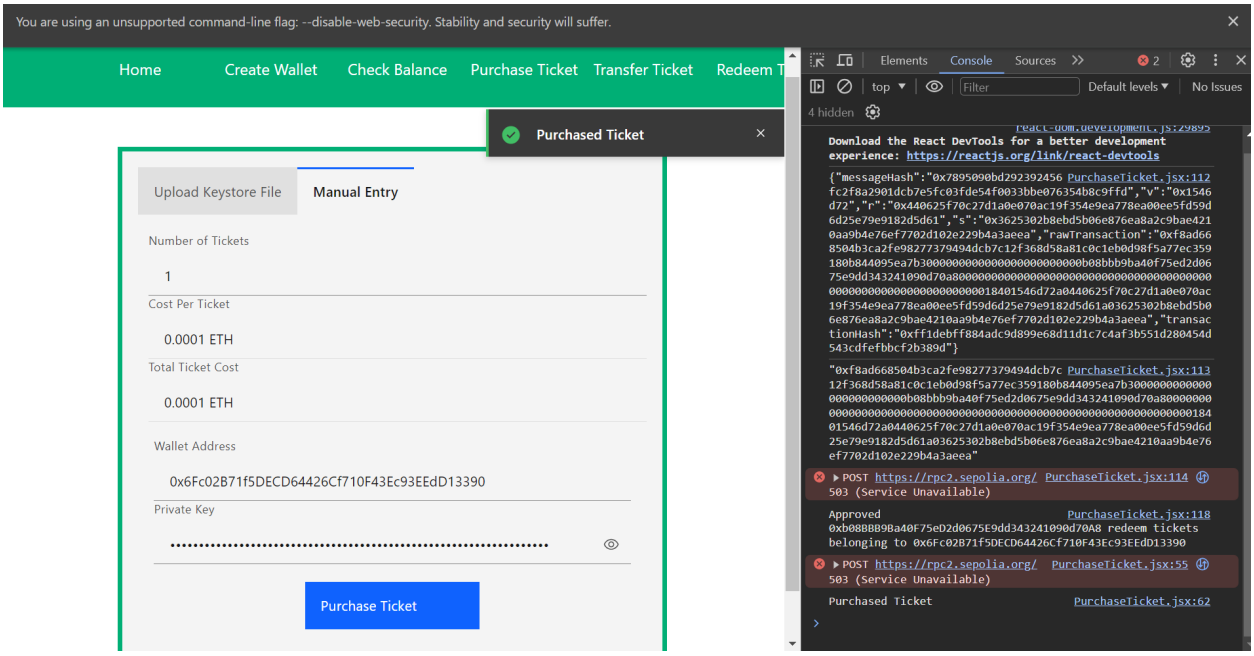
Contract deployment transaction hash:
0x5fd09df258092b5b852dbbe5259bc9ff2a32b31e813321e12fa39bc83c8f1dc7

Contract address: 0x94dCB7c12F368D58a81C0C1Eb0d98F5a77ec3591

Purchase of Ticket via Wallet Address and Private Key

Buy ticket transaction hash:
0xe33ee3d74bec23aeaa6aecbf121b7606e82936b7c6d8cb4624f282d5a1e4eaae

Approve transfer to doorman:
0x8abf56c7a3e60b2e12024d261c857d1f9edc5753cf374daefca6634d3b392352



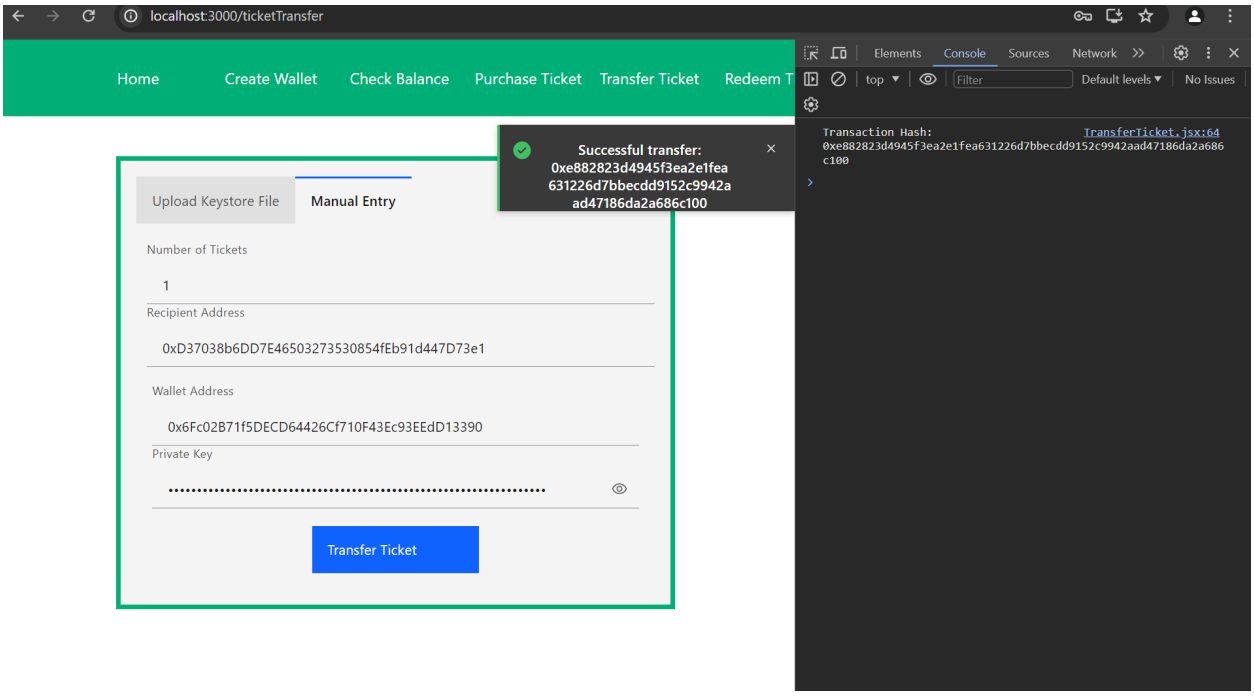
<u>Note</u>
The Sepholia endpoint fails due to CORs error often so most of my transactions were done using chrome.exe --user-data-dir="C://Chrome dev session" --disable-web-security. Also sometimes the network returns error despite successfully completing the transaction. In my case where I have both an approve and buyToken transaction executing, the Sepholia network sometimes falsely reports an error back from the approve transaction and this fails the buyToken transaction. If this is happening when you are demoing my webpage, please comment out where `'await approveTicketTransfer();'` is being called.

<u>Transfer of ticket from one user account to another</u>
This functionality can be used to manually transfer a ticket to a friend, back to the contract or to the doorman address.

Transfer transaction hash:
[0xe882823d4945f3ea2e1fea631226d7bbecdd9152c9942aad47186da2a686c100]

Sender address:
0x6Fc02B71f5DECD64426Cf710F43Ec93EEdD13390

Recipient address:
0xD37038b6DD7E46503273530854fEb91d447D73e1



Redeem ticket

The functionality of this page allows the doorman to redeem a ticket using the doormans private key. The doorman must enter his private key and the wallet address of the holder of the tickets. This allows the doorman to transfer the tickets to the doormans address. This means the ticket holder no longer has to give the doorman their private key when he is redeeming the tickets, as the ticket holder already signed off an approval of this when they bought the ticket.

TransferFrom transaction hash:
0x151c26f59365a2ead96d43fe57a657ef4199a3d875ffc7cfb8c46770536c1aa4

The doorman (0xb08BBB9Ba40F75eD2d0675E9dd343241090d70A8) created the transaction and signed off on the transaction, transferring 2 MatchTickets from the holder (0x6Fc02B71f5DECD64426Cf710F43Ec93EEdD13390) to the doormans wallet, marking them as redeemed. Feel free to change the doorman address in App.js when demoing, otherwise you would need the private key of the doorman wallet I created.

Note on tests, error handling

Tests

- I have included comprehensive testing on both the frontend with jest and the backend contract using truffle.

- Truffle tests can be found in \test\ticketing_system_contract_test.js. Truffle was difficult to set up for myself, so I will include a screenshot of the succeeding truffle tests run below in case you are also encountering difficulty.
- I have included instructions in the parents directory README on how to run the tests suite and at the bottom a resolution to errors.

```
PS C:\Users\johnm\OneDrive\ISE\EthicalHackingAndSecurity\BlockChain\BlockchainWeb3DApp\ticket-system-webapp> truffle test
Using network 'test'.


> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


  Contract: TicketingSystem
    ✓ Should return correct total supply
    ✓ Should return correct balance of deployer
    ✓ Should fail when not enough ETH is sent (186ms)
    ✓ Should pass when enough ETH is sent (44ms)
    ✓ Should only be able to buy a maximum of 99 tickets per transaction (55ms)
    ✓ Should transfer tokens if account has tokens, otherwise it should reject the transaction (76ms)
    ✓ Should only allow transferFrom transactions to go through if the owner has approved the transfer (121ms)


  7 passing (922ms)

PS C:\Users\johnm\OneDrive\ISE\EthicalHackingAndSecurity\BlockChain\BlockchainWeb3DApp\ticket-system-webapp>
```

<u>Error handling</u>

- Transaction calls are wrapped in try catch blocks.
- Notifications have been implemented on the webpage. They can be seen in App.js and the function is passed as a prop value into other pages.
- Confirmation boxes appear when creating transactions. In purchase, transfer and redeem ticket pages a confirmation popup will appear. This is to handle the case where a user clicks on a button by accident.
- Web3 libraries are used in CheckBalance to verify the addresses being checked are valid addresses. Line 20 of \pages\CheckBalance.jsx

Reference to libraries/documentation

- Truffle: https://archive.trufflesuite.com/docs/truffle/how-to/debug-test/test-your-contracts/
- React: https://react.dev/
- Carbon Components: https://react.carbondesignsystem.com/?path=/story/components-accordion--default
- Web3.js: https://web3js.readthedocs.io/